



Cloud Specialist
Unità Formativa (UF): Containers Microservizi
Serverless
Docente: Denis Maggiorotto
Titolo argomento: Creiamo immagini con Docker

Creare un container Docker con Node.js

An orange horizontal line with diamond-shaped arrowheads at both ends, positioned below the word "Docker" in the title.

Il Dockerfile

FROM *image*: Inizializza la build e setta l'immagine base sulla quale vengono eseguite le istruzioni che seguono.

Es: `FROM ubuntu:15.1`

RUN *command*: esegue un comando in un nuovo livello, sopra a quelli dell'immagine corrente e ne esegue la commit. L'immagine risultante viene usata dalle prossime istruzioni contenute nel Dockerfile.

Es: `RUN yum install httpd`

Il Dockerfile

EXPOSE *port*: informa Docker che il container è in ascolto sulle porte di rete specificate in fase di runtime. È possibile specificare se la porta è in ascolto su TCP o UDP e il valore predefinito è TCP se il protocollo non è specificato.

Es: EXPOSE 80
 EXPOSE 80/tcp

ENV *key[=value]*: Serve a settare una variabile d'ambiente che sarà disponibile all'interno del container. E' possibile effettuare l'override del valore dal comando `docker run`.

Es:

Il Dockerfile

COPY src dest: copia nuovi file o directory da <src> e li aggiunge al filesystem del container nel percorso <dest>

Es: `COPY app /app`

ADD src dest: copia nuovi file o directory da <src> e li aggiunge al filesystem del container nel percorso <dest>.

A differenza di COPY, con ADD src può essere una URL, in secondo luogo, è possibile estrarre un file tar dall'origine direttamente nella destinazione.

Es: `ADD http://myhost.com/file.txt /app`

Il Dockerfile

USER *user[:group]*: imposta il nome utente (o UID) e il gruppo utente (o GID) da utilizzare durante l'esecuzione dell'immagine e per tutte le istruzioni RUN, CMD ed ENTRYPOINT che lo seguono nel Dockerfile

Es: `USER oracle:oracle`

WORKDIR *dir*: imposta la directory di lavoro per tutte le istruzioni RUN, CMD, ENTRYPOINT, COPY e ADD che seguono nel Dockerfile

Es: `WORKDIR /app`

Il Dockerfile

ARG nome[=valore di default]: definisce una variabile che gli utenti possono passare in fase di compilazione con il comando `docker build` utilizzando il flag `--build-arg <varname>=<valore>`.

Es: ARG user=user1

Il Dockerfile

VOLUME path: crea un punto di mount con il nome specificato e lo contrassegna come contenente volumi montati esternamente. I volumi esterni verranno poi montati dall'istruzione `docker run`

Es: `VOLUME /myvol`

LABEL *chiave="valore"*: aggiunge metadati a un'immagine. Una LABEL è una coppia chiave-valore.

Es: `LABEL version="1.0"`

Il Dockerfile

ENTRYPOINT *command param1 param2*: Consente di configurare il processo che verrà eseguito dentro al container. Questo processo non potrà essere modificato dal comando `docker run`.

Es: `ENTRYPOINT /usr/sbin/apache2ctl -D FOREGROUND`

CMD *command param1 param2*: Consente di configurare il processo che verrà eseguito dentro al container. Questo processo potrà essere modificato dal comando `docker run`.

Es: `CMD /usr/sbin/apache2ctl -D FOREGROUND`

II Dockerfile

Dockerfile reference

<https://docs.docker.com/engine/reference/builder/>

LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 06 – Node.js containers

Creare un container Docker con Java



LAB

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Lab 07 – Java containers

Assignment

https://github.com/sunnyvale-academy/ITS-ICT_Containers

Assignment 01 – Create a Redis server image